JavaScript Interview Notes

Fast Track for React + E-Commerce Frontend Interviews

Purpose

Concise notes for the JavaScript concepts needed before building React Login, Registration and Product pages. Focus: concept clarity, React usage, examples and interview-ready answers.

1. var, let and const

Keywords used to declare variables. Use **const** by default, **let** when reassignment is required, and generally avoid **var** in modern React.

Feature	var	let	const
Scope	Function	Block	Block
Reassignment	Yes	Yes	No
Redeclaration	Yes	No	No

```
const name = "Lakshmi";
let count = 0;
count = 1;
```

> **Interview:** var is function-scoped and allows redeclaration/reassignment. let and const are block-scoped; let allows reassignment while const does not.

2. Scope, Hoisting and TDZ

Scope determines where a variable can be accessed. **Hoisting** describes declaration processing before execution. var is initialized with undefined; let/const are hoisted but cannot be accessed before declaration because of the **Temporal Dead Zone**.

```
console.log(a); // undefined
var a = 10;

// console.log(b); // ReferenceError
let b = 20;
```

> **Interview:** let and const are hoisted, but remain in the TDZ until their declaration is executed.

3. Functions and Arrow Functions

A **function** is reusable code that performs a task. An **arrow function** is concise ES6 function syntax. Arrow functions do not have their own this; they use lexical this.

```
function add(a, b) {
return a + b;
}

const addArrow = (a, b) => a + b;
```

> **React usage:** Components, event handlers, map/filter callbacks and hook callbacks commonly use arrow functions.

4. Destructuring

Destructuring extracts values from arrays or properties from objects. Object destructuring is property-name based; array destructuring is position based.

```
const user = { name: "Lakshmi", role: "ADMIN" };
const { name, role } = user;
```

```
const [email, setEmail] = useState("");
```

React usage: Props and useState use destructuring frequently.

5. Spread and Rest (...)

Spread expands array/object values. **Rest** collects remaining values. They use the same ... syntax but do opposite jobs.

Operator	Meaning	Typical use
Spread	Expand	Copy/update arrays or objects
Rest	Collect	Collect remaining values

```
const updatedUser = {
...user,
role: "USER"
};

const [first, ...remaining] = numbers;
```

React usage: Spread is commonly used to create a new state object instead of mutating existing state.

6. map, filter, find and reduce

map transforms every element; filter returns matching elements; find returns the first match; reduce combines elements into one accumulated result.

Method	Purpose	Returns
map	Transform every element	New array
filter	Select matches	New array
find	First match	Element / undefined
reduce	Combine values	Accumulated result

```
const names = products.map(p => p.name);
const costly = products.filter(p => p.price > 1000);
const item = products.find(p => p.id === 2);
const total = prices.reduce((sum, p) => sum + p, 0);
```

React usage: map renders products; filter supports search; find locates a product; reduce can calculate totals.

7. Promises and async/await

A **Promise** represents eventual completion or failure of asynchronous work. States: pending, fulfilled, rejected. async/await provides readable syntax for consuming Promises.

```
const getProducts = async () => {
try {
const response = await fetch("/api/products");
const data = await response.json();
} catch (error) {
console.log(error);
}
};
```

Project usage: Registration POST, Login POST and Product API calls. await does not freeze the whole browser.

8. ES6 Modules - import/export

Modules split code into reusable files. export exposes functionality; import consumes it. One default export per module; multiple named exports are allowed.

Type	Export	Import
Default	export default Login	import Login from './Login'
Named	export const login = ...	import { login } from './api'

```
export default Login;
import Login from "./Login";

export const login = () => {};
import { login } from "./api";
```

React usage: Components, API services, contexts and custom hooks live in separate modules.

9. Closures

A closure occurs when an inner function retains access to variables from its outer lexical scope even after the outer function has completed.

```
function outer() {
let count = 0;
return () => ++count;
}
const counter = outer();
```

Interview: A closure is a function together with access to its surrounding lexical environment.

10. Callback Functions

A callback is a function passed to another function so the receiving function can execute it.

```
products.map(product => product.name);

// React:
onClick={handleLogin}
```

Important: onClick={handleLogin} passes the function. onClick={handleLogin()} invokes it during rendering.

11. Event Loop - Basics

JavaScript's main execution is single-threaded. The host environment enables asynchronous behavior; the event loop coordinates eligible queued work with the call stack.

```
console.log("Start");
setTimeout(() => console.log("Async"), 0);
console.log("End");
// Start, End, Async
```

Interview: JavaScript is synchronous/single-threaded at its core, with asynchronous behavior supported by the environment, queues, callbacks/Promises and event loop.

12. Template Literals

Template literals use backticks and ${...} for string interpolation and can span multiple lines.

```
const productId = 101;
const url = `/api/products/${productId}`;
```

Project usage: Dynamic API URLs and readable strings.

13. Optional Chaining (?.)

Optional chaining safely accesses nested properties. If the value before ?. is null or undefined, the expression returns undefined rather than failing on that property access.

```
const city = user?.address?.city;
const name = product?.name;
```

> **React usage:** Useful when API data is initially unavailable or nested fields are optional.

14. Nullish Coalescing (??)

?? supplies a fallback only when the left value is null or undefined. Unlike ||, valid falsy values such as 0 are preserved.

```
const stock = product.stock ?? 0;

0 ?? 10; // 0
0 || 10; // 10
```

> **Interview:** ?? checks null/undefined; || reacts to any falsy value.

15. == vs ===

== uses loose equality and may perform type coercion. === uses strict equality and does not make different types equal through coercion. Prefer === and !==.

```
5 == "5"; // true
5 === "5"; // false
```

> **Interview:** Strict equality is generally preferred because it avoids surprising coercion.

Quick Revision - 15 Interview Questions

1. Difference between var, let and const?
var is function-scoped; let/const are block-scoped. let can be reassigned; const cannot.

2. Are let and const hoisted?
Yes, but they remain in the TDZ until declaration.

3. What is an arrow function?
Concise ES6 function syntax; it has no own this.

4. Parameter vs argument?
Parameter is defined by a function; argument is the actual supplied value.

5. What is destructuring?
Concise extraction of object properties or array values into variables.

6. Spread vs rest?
Spread expands; rest collects.

7. map vs forEach?
map creates a transformed array; forEach normally performs an action and does not create that result array.

8. filter vs find?
filter returns all matches in an array; find returns the first match or undefined.

9. What is a Promise?
An object representing eventual completion or failure of asynchronous work.

10. Promise states?
Pending, fulfilled and rejected.

11. What is async/await?
Cleaner syntax for working with Promises.

12. What is a closure?
A function retaining access to its surrounding lexical scope.

13. What is a callback?
A function passed to another function to be invoked by it.

14. What is the event loop?
It coordinates eligible queued asynchronous work with the call stack.

15. == vs ===?
== may coerce types; === uses strict equality without coercive equality.

React Project Connection

Registration: useState, destructuring, spread, events, async/await, API calls.
Login: forms, callbacks, async/await, JWT handling, routing and Context.
Products: useEffect, map, filter, find, props, optional chaining and REST API integration.

Next: React fundamentals -> Registration -> Login/Auth -> Protected Routes -> Products -> Context/Redux -> optimization -> lazy loading/testing -> interview revision.